Polymorphism and Inheritance in Java

1 Advantages of Polymorphism

1.1 Increase Program Flexibility

We can use a parent type (`Media`) to manage many child types such as `Book`, `DVD`, and `CD`.

Example:

```
ArrayList<Media>
```

This list can contain different types of media objects.

We do not need separate lists such as:

```
ArrayList<Book>
ArrayList<DVD>
ArrayList<CD>
```

1.2 Easy to Extend the Program

Later, if we add a new class:

```
class Magazine extends Media
```

we do not need to modify old code.

We only need to override `toString()` and the program will continue to work correctly.

This follows the Open/Closed Principle:

- Open for extension
- Closed for modification

1.3 Reduce Dependency Between Classes

The code works with the parent class (`Media`) instead of depending directly on child classes.

This makes the program:

- easier to maintain
- easier to upgrade
- easier to reuse

1.4 Cleaner and Shorter Code

We do not need statements such as:

```
if(media instanceof Book)
```

or:

```
if(media instanceof DVD)
```

Java automatically selects the correct method to execute.

2 Inheritance Useful to Achieve Polymorphism in Java

Inheritance is the foundation of Polymorphism.

To achieve polymorphism, child classes must inherit from a parent class.

Example:

```
class Book extends Media
class DVD extends Media
class CD extends Media
```

Because of inheritance:

- Book
- DVD
- CD

can all be treated as a `Media` object.

Example:

```
Media media = new Book(...);
```

This is called **Upcasting**.

Then, child classes override methods such as:

```
@Override
public String toString()
```

When calling:

```
media.toString();
```

Java determines the real object type and calls the correct method automatically.

This is Polymorphism.

Summary

Inheritance provides:

- IS-A relationship
- ability to use a common parent type

Polymorphism provides:

- ability for methods to behave differently depending on the real object type

These two concepts work together.

3 Differences Between Polymorphism and Inheritance in Java

Inheritance	**Polymorphism**
Mechanism for inheriting properties and methods	Ability of a method to behave in multiple ways
Uses the `extends` keyword	Usually uses method overriding
Creates parent-child relationships	Creates flexible behavior
Focuses on class structure	Focuses on behavior
Helps reuse code	Helps make programs flexible
Example: `Book extends Media`	Example: `media.toString()` behaves differently

4 Simple Example

4.1 Inheritance

```
class Animal {
    void sound() {}
}

class Dog extends Animal {
}
```

`Dog` inherits from `Animal`.

4.2 Polymorphism

```
class Dog extends Animal {
    void sound() {
        System.out.println("Woof");
    }
}

class Cat extends Animal {
    void sound() {
        System.out.println("Meow");
    }
}

Animal a = new Dog();
a.sound();
```

Output:

```
Woof
```

If:

```
a = new Cat();
a.sound();
```

Output:

```
Meow
```

The same method call:

```
a.sound()
```

produces different results depending on the actual object type. This is called **Polymorphism**.